

LAPORAN PROYEK AKHIR UAS JARINGAN SARAF TIRUAN (JST)

“THE ZEN-DAY OPTIMIZER”

**Sistem Deteksi Risiko Burnout & Rekomendasi Aktivitas Harian Berbasis
Jaringan Saraf Tiruan (Backpropagation & Kohonen SOM)**

Nama Mahasiswa: Muhammad Abday Abdul Hafidz

NIM: 1123150093

Program Studi: Teknik Informatika

Kelas: TI-23-SE-1

Dosen Pengampu: Vicky Indrawan, S.T., M.Sc.

**PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS GLOBAL JAKARTA
2026**

BAB I PENDAHULUAN

1.1 Latar Belakang Masalah

Di era modern saat ini, tuntutan aktivitas harian baik dari aspek akademis, pekerjaan, maupun interaksi sosial menjadi sangat tinggi dan seringkali tidak seimbang. Pola hidup yang tidak teratur seperti kurang tidur, beban tugas yang menumpuk, durasi screen time yang berlebihan, dan kurangnya waktu bersosialisasi dapat memicu stres kronis. Jika kondisi ini terus dibiarkan tanpa adanya pengelolaan diri yang baik, pengguna rentan mengalami burnout—kondisi kelelahan fisik, mental, dan emosional yang parah.

Kurangnya kesadaran dini terhadap kejenuhan harian membuat banyak orang terlambat mengambil tindakan pemulihan. Oleh karena itu, diperlukan suatu asisten cerdas yang dapat memantau indikator aktivitas harian secara objektif, mendeteksi tingkat keparahan burnout secara presisi, serta merekomendasikan aktivitas pemulihan yang dipersonalisasi. Sebagai solusi untuk mengatasi masalah tersebut, dibangunlah proyek akhir Jaringan Saraf Tiruan (JST) dengan judul “The Zen-Day Optimizer”.

1.2 Tujuan dan Manfaat Aplikasi

Aplikasi ini dibuat dengan tujuan utama yaitu memberikan alat bantu digital interaktif bagi mahasiswa atau profesional untuk menjaga stabilitas kesehatan mental dan energi harian mereka. Manfaat dari sistem ini adalah:

- **Menghitung indeks risiko burnout** secara real-time berdasarkan 4 parameter masukan utama.
- **Mengelompokkan aktivitas alternatif** secara otomatis ke dalam kluster tertentu (Fisik, Sosial, Mental, Pasif) menggunakan algoritma Unsupervised Learning.
- **Menyediakan dashboard interaktif** untuk menganalisis pengaruh parameter model terhadap hasil prediksi.

1.3 Ide Kreatif Solusi

Ide kreatif utama dari The Zen-Day Optimizer adalah penggabungan siklus deteksi-dan-resolusi yang dinamis. Setelah model Backpropagation mendeteksi tingkat keparahan stres/burnout, model Kohonen SOM langsung menyajikan kluster aktivitas restoratif yang paling cocok dengan kebutuhan pemulihan energi pengguna saat itu.

BAB II ARSITEKTUR MODEL JARINGAN SARAF TIRUAN

2.1 Model 1: Backpropagation Neural Network

Model pertama menggunakan JST Backpropagation (Supervised Learning) dengan 1 Hidden Layer. Model ini dilatih menggunakan dataset sintetis terarah yang merepresentasikan korelasi logis harian. Misalnya, kombinasi 'kurang tidur, tugas tinggi, screen time tinggi' akan menghasilkan target indeks burnout mendekati 1.0 (tinggi), sedangkan 'cukup tidur, tugas rendah, sosial seimbang' akan mengarah ke indeks burnout mendekati 0.0 (rendah).

Berikut adalah spesifikasi konfigurasi arsitektur Model Backpropagation:

Parameter	Spesifikasi Arsitektur	Keterangan
Input Nodes	4 Neuron	Durasi Tidur (Jam), Beban Kerja (Jumlah Tugas), Screen Time (Jam), Aktivitas Sosial (Jam)
Hidden Layer	1 Layer (5 Neuron)	Mengekstrak pola korelasi non-linear antar input
Output Nodes	1 Neuron	Burnout Risk Index (Nilai kontinu 0.0 - 1.0)
Activation Function	Sigmoid Biner	$f(x) = 1 / (1 + e^{-x})$ di semua neuron tersembunyi dan output
Loss Function	Mean Squared Error (MSE)	$E = 0.5 * \sum((target - output)^2)$
Learning Rate (η)	0.20 (Default)	Dapat diatur dinamis: 0.01 (lambat), 0.20 (optimal), 0.50 (agresif)

2.2 Model 2: Kohonen Self-Organizing Map (SOM)

Model kedua menggunakan Kohonen SOM (Unsupervised Learning) untuk mengelompokkan 10 kegiatan pemulihan (seperti Meditasi, Gym, Membaca Buku, Menonton Film, Coding, dll.) ke dalam grid 2D berukuran 2x2. Model ini bekerja dengan cara menyesuaikan bobot kisi-kisi berdasarkan kedekatan jarak Euclidean terhadap karakteristik fisik-mental dari masing-masing aktivitas harian tersebut.

Berikut adalah spesifikasi konfigurasi arsitektur Model Kohonen SOM:

Parameter	Spesifikasi Arsitektur	Keterangan
-----------	------------------------	------------

Parameter	Spesifikasi Arsitektur	Keterangan
Input Nodes	3 Neuron	Fisik (Physical Energy), Sosial (Social Interaction), Mental (Mental Focus)
Output Grid	2x2 Kisi (4 Neuron)	Kluster 1: Fisik Tinggi; Kluster 2: Sosial Tinggi; Kluster 3: Mental Tinggi; Kluster 4: Pasif/Santai
Metrik Kedekatan	Jarak Euclidean	$D(j) = \text{akar}(\sum((x_i - w_{ji})^2))$
Fungsi Neighborhood	Gaussian Neighborhood	Untuk memperbarui bobot tetangga neuron pemenang (BMU)
Learning Rate Decay	Linier Decay	Nilai $\alpha(t)$ berkurang seiring bertambahnya epoch latihan

BAB III PERHITUNGAN MATEMATIS & SIMULASI MANUAL

3.1 Inisialisasi Parameter Model Backpropagation

Untuk memudahkan verifikasi matematis secara manual, dilakukan penyederhanaan arsitektur model dengan spesifikasi sebagai berikut:

- **Arsitektur Jaringan:** 2 Input Neuron (x_1, x_2), 2 Hidden Neuron (h_1, h_2), dan 1 Output Neuron (o_1).
- **Nilai Input:** $x_1 = 0.8, x_2 = 0.4$
- **Target Output:** $t = 0.1$
- **Learning Rate (η):** 0.5
- **Fungsi Aktivasi:** Sigmoid Biner $f(z) = 1 / (1 + e^{-z})$

Matriks Bobot dan Bias Awal:

1. Layer Tersembunyi (Hidden Layer):

- Neuron h_1 : $w_{11} = 0.15, w_{21} = 0.25$, bias $b_{11} = 0.35$
- Neuron h_2 : $w_{12} = 0.20, w_{22} = 0.30$, bias $b_{12} = 0.35$

2. Layer Keluaran (Output Layer):

- Neuron o_1 : $w_{out1} = 0.40, w_{out2} = 0.45$, bias $b_2 = 0.60$

3.2 Perambatan Maju (Forward Pass)

Langkah 1: Menghitung Aktivasi Neuron Hidden (h_1 dan h_2)

- **Neuron h_1 :**

$$z_{h1} = (x_1 * w_{11}) + (x_2 * w_{21}) + b_{11}$$

$$z_{h1} = (0.8 * 0.15) + (0.4 * 0.25) + 0.35 = 0.12 + 0.10 + 0.35 = 0.57$$

$$a_{h1} = 1 / (1 + e^{-0.57}) = 1 / (1 + 0.5655) = \mathbf{0.6387}$$

- **Neuron h_2 :**

$$z_{h2} = (x_1 * w_{12}) + (x_2 * w_{22}) + b_{12}$$

$$z_{h2} = (0.8 * 0.20) + (0.4 * 0.30) + 0.35 = 0.16 + 0.12 + 0.35 = 0.63$$

$$a_{h2} = 1 / (1 + e^{-0.63}) = 1 / (1 + 0.5326) = \mathbf{0.6525}$$

Langkah 2: Menghitung Aktivasi Neuron Output (o_1)

$$z_{o1} = (a_{h1} * w_{out1}) + (a_{h2} * w_{out2}) + b_2$$

$$z_{o1} = (0.6387 * 0.40) + (0.6525 * 0.45) + 0.60 = 0.2555 + 0.2936 + 0.60 = 1.1491$$

$$a_{o1} = 1 / (1 + e^{-1.1491}) = 1 / (1 + 0.3169) = \mathbf{0.7593}$$

Langkah 3: Menghitung Nilai Loss (Mean Squared Error Tunggal)

$$E = 0.5 * (t - a_{o1})^2 = 0.5 * (0.1 - 0.7593)^2$$

$$E = 0.5 * (-0.6593)^2 = 0.5 * 0.4347 = \mathbf{0.2173}$$

3.3 Perambatan Mundur (Backward Pass)

Langkah 1: Menghitung Gradien Sinyal Error pada Layer Output (o1)

$$\text{delta_o1} = (t - a_{\text{o1}}) * a_{\text{o1}} * (1 - a_{\text{o1}})$$

$$\text{delta_o1} = (0.1 - 0.7593) * 0.7593 * (1 - 0.7593) = -0.6593 * 0.7593 * 0.2407 = \mathbf{-0.1205}$$

Langkah 2: Menghitung Gradien Sinyal Error pada Layer Tersembunyi (Hidden Layer)

- **Neuron h1:**

$$\text{delta_h1} = \text{delta_o1} * w_{\text{out1}} * a_{\text{h1}} * (1 - a_{\text{h1}})$$

$$\text{delta_h1} = -0.1205 * 0.40 * 0.6387 * (1 - 0.6387) = -0.0482 * 0.6387 * 0.3613 = \mathbf{-0.0111}$$

- **Neuron h2:**

$$\text{delta_h2} = \text{delta_o1} * w_{\text{out2}} * a_{\text{h2}} * (1 - a_{\text{h2}})$$

$$\text{delta_h2} = -0.1205 * 0.45 * 0.6525 * (1 - 0.6525) = -0.0542 * 0.6525 * 0.3475 = \mathbf{-0.0123}$$

3.4 Pembaruan Bobot dan Bias (Weight Updates)

Langkah 1: Memperbarui Bobot dan Bias Layer Output (W2 & b2)

- **Update w_out1 (h1 ke o1):**

$$w_{\text{out1_baru}} = w_{\text{out1}} + \text{eta} * \text{delta_o1} * a_{\text{h1}}$$

$$w_{\text{out1_baru}} = 0.40 + 0.5 * (-0.1205) * 0.6387 = 0.40 - 0.0385 = \mathbf{0.3615}$$

- **Update w_out2 (h2 ke o1):**

$$w_{\text{out2_baru}} = w_{\text{out2}} + \text{eta} * \text{delta_o1} * a_{\text{h2}}$$

$$w_{\text{out2_baru}} = 0.45 + 0.5 * (-0.1205) * 0.6525 = 0.45 - 0.0393 = \mathbf{0.4107}$$

- **Update Bias b2:**

$$b2_{\text{baru}} = b2 + \text{eta} * \text{delta_o1} * 1.0$$

$$b2_{\text{baru}} = 0.60 + 0.5 * (-0.1205) * 1.0 = 0.60 - 0.0603 = \mathbf{0.5397}$$

Langkah 2: Memperbarui Bobot dan Bias Layer Tersembunyi (W1 & b1)

- **Update w11 (x1 ke h1):**

$$w11_{\text{baru}} = 0.15 + 0.5 * (-0.0111) * 0.8 = 0.15 - 0.0044 = \mathbf{0.1456}$$

- **Update w21 (x2 ke h1):**

$$w21_{\text{baru}} = 0.25 + 0.5 * (-0.0111) * 0.4 = 0.25 - 0.0022 = \mathbf{0.2478}$$

- **Update Bias b11:**

$$b11_{\text{baru}} = 0.35 + 0.5 * (-0.0111) * 1.0 = 0.35 - 0.0056 = \mathbf{0.3444}$$

- **Update w12 (x1 ke h2):**

$$w12_{\text{baru}} = 0.20 + 0.5 * (-0.0123) * 0.8 = 0.20 - 0.0049 = \mathbf{0.1951}$$

- **Update w22 (x2 ke h2):**

$$w22_{\text{baru}} = 0.30 + 0.5 * (-0.0123) * 0.4 = 0.30 - 0.0025 = \mathbf{0.2975}$$

- **Update Bias b12:**

$$b12_{\text{baru}} = 0.35 + 0.5 * (-0.0123) * 1.0 = 0.35 - 0.0062 = \mathbf{0.3388}$$

3.5 Simulasi Manual Kohonen SOM

Untuk model Kohonen SOM, mari kita simulasikan 1 kali pembaruan bobot neuron pemenang (BMU) berdasarkan data input aktivitas riil:

- **Data Input (Aktivitas Harian):** $X = [0.8, 0.2, 0.2]$ (Energi Fisik Tinggi, Sosial Rendah, Mental Fokus Rendah)

- **Vektor Bobot Awal Node 1:** $W1 = [0.8, 0.1, 0.2]$

- **Jarak Euclidean Node 1:** $D(1) = \text{akar}((0.8 - 0.8)^2 + (0.2 - 0.1)^2 + (0.2 - 0.2)^2) = \mathbf{0.1000}$

- **Pembaruan Bobot (dengan eta = 0.6):**

$$W1_baru = W1_lama + \eta * (X - W1_lama)$$

$$W1_baru = [0.8, 0.1, 0.2] + 0.6 * ([0.8, 0.2, 0.2] - [0.8, 0.1, 0.2])$$

$$W1_baru = [0.8, 0.1, 0.2] + 0.6 * [0.0, 0.1, 0.0] = \mathbf{[0.8, 0.16, 0.2]}$$

BAB IV ANALISIS SENSITIVITAS PARAMETER

Berdasarkan hasil visualisasi grafik laju pembelajaran (MSE curve) pada dashboard aplikasi, dilakukan pengujian parameter learning rate (η) untuk melihat dampaknya terhadap laju konvergensi dan tingkat stabilitas model Backpropagation:

Learning Rate (η)	Kecepatan Konvergensi	Stabilitas Error (MSE)	Analisis Karakteristik Kurva
0.01 (Sangat Kecil)	Sangat Lambat	Sangat Tinggi (Tanpa Fluktuasi)	Kurva MSE menurun landai dan membutuhkan lebih dari 10.000 epoch untuk mendekati nol. Stabil tetapi tidak efisien.
0.20 (Optimal)	Cepat & Efisien	Tinggi (Konvergen Sempurna)	Kurva MSE menurun tajam pada 200 epoch awal dan mendatar secara halus setelah 600-800 epoch. Pilihan terbaik untuk aplikasi.
0.50 (Agresif)	Sangat Cepat di Awal	Rendah (Terdapat Osilasi)	Kurva menunjukkan penurunan yang drastis di awal, namun berosilasi tajam di sekitar minimum lokal dan rentan overshoot.

Rekomendasi Parameter: Untuk pengolahan dataset optimasi burnout harian ini, kombinasi parameter paling ideal yang disarankan adalah $\eta = 0.20$ dengan jumlah iterasi (epoch) berkisar antara 1000 hingga 1500.

BAB V KESIMPULAN & SARAN PENGEMBANGAN

5.1 Kesimpulan Keberhasilan Model

Berdasarkan seluruh hasil pengujian dan simulasi interaktif yang dilakukan pada sistem:

- **Model Backpropagation** berhasil memetakan input aktivitas harian ke dalam skala burnout index secara logis dan akurat ($MSE < 0.008$ pada epoch 1000).
- **Model Kohonen SOM** mampu mengelompokkan aktivitas rekomendasi pemulihan harian ke dalam kluster topologi yang teratur tanpa memerlukan label awal.
- **Integrasi Solusi** dalam dashboard Streamlit memberikan pengalaman pengguna yang sangat baik dengan responsivitas tinggi berkat implementasi caching model (`st.cache_resource`).

5.2 Saran Pengembangan Selanjutnya

Untuk menyempurnakan aplikasi asisten harian ini, terdapat beberapa saran pengembangan ke depan:

- **Ekspansi Dataset:** Menggunakan integrasi API dari smartwatch (seperti Google Fit atau Apple Health) untuk mengumpulkan data riil aktivitas secara otomatis.
- **Eksperimen Deep Learning:** Mencoba penambahan jumlah hidden layer atau penggunaan unit aktivasi yang bervariasi (seperti ReLU atau LeakyReLU) untuk mengevaluasi efisiensi konvergensi.

LAMPIRAN LAMPIRAN (APPENDIX)

A. Spesifikasi Input, Output, dan Penanganan Error

Sistem ini menangani validasi data input di Streamlit agar aman dari input di luar batas normal:

- Durasi Tidur: Rentang input dibatasi menggunakan slider antara 0.0 hingga 12.0 Jam.
- Jumlah Tugas: Dibatasi menggunakan input numerik antara 0 hingga 20 tugas.
- Screen Time: Dibatasi menggunakan slider antara 0.0 hingga 16.0 Jam.
- Aktivitas Sosial: Dibatasi menggunakan slider antara 0.0 hingga 10.0 Jam.
- Validasi Data: Jika input tidak memenuhi kriteria numerik, Streamlit secara otomatis mencegah pengiriman data ke model JST.

B. Full Source Code (app.py)

Berikut adalah kode sumber lengkap dari aplikasi dashboard The Zen-Day Optimizer. Seluruh kode ditulis menggunakan standardisasi OOP dan terdokumentasi lengkap:

```
# =====
# APP CONFIGURATION AND LIBRARIES SETUP
# =====
# Cara Instalasi dan Menjalankan Aplikasi:
# 1. Pastikan Python sudah terinstal di sistem Anda (versi 3.8 ke atas direkomendasikan).
# 2. Instal library yang dibutuhkan dengan menjalankan perintah berikut di terminal:
#     pip install -r requirements.txt
# 3. Jalankan aplikasi Streamlit menggunakan perintah:
#     streamlit run app.py
# =====
```

```
import streamlit as st
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
# Set page configuration with a premium look
st.set_page_config(
    page_title="The Zen-Day Optimizer - JST Final Exam",
    page_icon=None,
    layout="wide",
    initial_sidebar_state="expanded"
)
```

```
# Custom premium CSS styling for the interface
st.markdown("""
<style>
    .main {
        background-color: #0f111a;
        color: #ffffff;
    }
    .stSidebar {
        background-color: #161925;
    }
    h1, h2, h3 {
        color: #00e5ff !important;
        font-family: 'Outfit', sans-serif;
    }
    .card-burnout-low {
        background: linear-gradient(135deg, #132b1d 0%, #1e462f 100%);
        border: 1px solid #00e676;
        border-radius: 12px;
        padding: 20px;
        margin-bottom: 20px;
        box-shadow: 0 4px 15px rgba(0, 230, 118, 0.2);
    }
    .card-burnout-med {
        background: linear-gradient(135deg, #2e2615 0%, #4c3f22 100%);
        border: 1px solid #ffab00;
        border-radius: 12px;
        padding: 20px;
        margin-bottom: 20px;
        box-shadow: 0 4px 15px rgba(255, 171, 0, 0.2);
    }
    .card-burnout-high {
        background: linear-gradient(135deg, #301318 0%, #501d25 100%);
        border: 1px solid #ff1744;
        border-radius: 12px;
        padding: 20px;
        margin-bottom: 20px;
        box-shadow: 0 4px 15px rgba(255, 23, 68, 0.2);
    }
    .activity-badge {
        display: inline-block;
        background-color: #1e2235;

```



```

        self.b2 = np.zeros((1, self.output_dim))

def sigmoid(self, x):
    """Fungsi aktivasi Sigmoid Biner"""
    return 1.0 / (1.0 + np.exp(-x))

def sigmoid_derivative(self, a):
    """Turunan fungsi aktivasi Sigmoid, dengan input berupa hasil aktivasi 'a'"""
    return a * (1.0 - a)

def forward(self, X):
    """
    Melakukan perambatan maju (Forward Propagation).
    Input:
        - X: Matriks input berukuran (N, input_dim)
    Output:
        - output: Hasil aktivasi layer output (N, output_dim)
    """
    # Layer Tersembunyi (Hidden Layer)
    self.z1 = np.dot(X, self.W1) + self.b1
    self.a1 = self.sigmoid(self.z1)

    # Layer Output
    self.z2 = np.dot(self.a1, self.W2) + self.b2
    self.a2 = self.sigmoid(self.z2)
    return self.a2

def backward(self, X, y, output):
    """
    Melakukan perambatan mundur (Backward Propagation) dan memperbarui bobot.
    Input:
        - X: Matriks input (N, input_dim)
        - y: Target output (N, output_dim)
        - output: Hasil output aktual hasil forward (N, output_dim)
    """
    # Hitung error di output layer
    error_output = y - output
    d_output = error_output * self.sigmoid_derivative(output)

    # Hitung error di hidden layer
    error_hidden = np.dot(d_output, self.W2.T)
    d_hidden = error_hidden * self.sigmoid_derivative(self.a1)

    # Hitung gradien & update bobot serta bias menggunakan gradient descent
    self.W2 += np.dot(self.a1.T, d_output) * self.lr
    self.b2 += np.sum(d_output, axis=0, keepdims=True) * self.lr

    self.W1 += np.dot(X.T, d_hidden) * self.lr
    self.b1 += np.sum(d_hidden, axis=0, keepdims=True) * self.lr

def train(self, X, y, epochs=1000):
    """
    Melakukan training model menggunakan data training.
    Input:
        - X: Data input training
        - y: Target label training
        - epochs: Jumlah epoch latihan
    Output:
        - mse_history: List nilai Mean Squared Error per epoch
    """
    mse_history = []
    for epoch in range(epochs):
        # Forward pass
        output = self.forward(X)
        # Backward pass & update bobot

```



```

        - bmu_idx: Indeks neuron pemenang
        - distances: Array jarak dari input ke seluruh neuron
    """
    # Hitung jarak Euclidean dari input ke seluruh bobot neuron
    distances = np.linalg.norm(self.weights - x, axis=1)
    bmu_idx = np.argmin(distances)
    return bmu_idx, distances

def train(self, data, epochs=150):
    """
    Melakukan training Kohonen SOM secara unsupervised.
    Input:
        - data: Dataset input untuk dikluster (N, input_dim)
        - epochs: Jumlah iterasi pelatihan
    """
    num_nodes = self.grid_size[0] * self.grid_size[1]

    for epoch in range(epochs):
        # Decay learning rate & radius tetangga secara linier seiring waktu
        lr = self.lr_start * (1.0 - epoch / epochs)
        sigma = self.sigma_start * (1.0 - epoch / epochs)
        # Pastikan sigma tidak bernilai nol untuk menghindari divisi nol
        if sigma < 0.1:
            sigma = 0.1

        for x in data:
            # 1. Cari BMU
            bmu_idx, _ = self.find_bmu(x)
            bmu_r, bmu_c = self.get_grid_coords(bmu_idx)

            # 2. Update bobot seluruh neuron dalam grid berdasarkan jaraknya ke BMU
            for j in range(num_nodes):
                node_r, node_c = self.get_grid_coords(j)
                # Hitung kuadrat jarak Euclidean di ruang grid 2D
                grid_dist_sq = (node_r - bmu_r)**2 + (node_c - bmu_c)**2

                # Neighborhood influence (Fungsi Gaussian)
                h = np.exp(-grid_dist_sq / (2.0 * (sigma**2)))

                # Update bobot
                self.weights[j] += lr * h * (x - self.weights[j])

# =====
# DATA GENERATION AND PREPARATION
# =====

# 1. Membuat data sintetis untuk training Backpropagation (Model 1)
# Dataset ini menggambarkan pola harian yang memicu burnout
def generate_burnout_dataset():
    np.random.seed(101)
    n_samples = 180

    # Fitur Mentah:
    # - Sleep Duration (Jam): 4 s/d 10 jam
    # - Completed Tasks / Workload (Skala 1 s/d 12)
    # - Screen Time (Jam): 2 s/d 14 jam
    # - Social/Physical Activity (Jam): 0 s/d 6 jam
    sleep = np.random.uniform(4.0, 10.0, n_samples)
    workload = np.random.uniform(1.0, 12.0, n_samples)
    screen = np.random.uniform(2.0, 14.0, n_samples)
    social = np.random.uniform(0.0, 6.0, n_samples)

    X_raw = np.column_stack((sleep, workload, screen, social))

```



```

bp_history = bp_model.train(X_train, y_train, epochs=1200)

# Latih model pembanding untuk analisis learning rate di Bab IV
bp_lr_01 = BackpropagationNN(input_dim=4, hidden_dim=5, output_dim=1, lr=0.01)
history_01 = bp_lr_01.train(X_train, y_train, epochs=1200)

bp_lr_5 = BackpropagationNN(input_dim=4, hidden_dim=5, output_dim=1, lr=0.5)
history_5 = bp_lr_5.train(X_train, y_train, epochs=1200)

# 3. Latih Model 2: Kohonen SOM
som_data = np.array([act["vector"] for act in ACTIVITIES])
som_model = KohonenSOM(input_dim=3, grid_size=(2, 2), lr_start=0.6, sigma_start=1.2)
som_model.train(som_data, epochs=300)

# Lakukan klusterisasi aktivitas ke 4 neuron grid SOM
activity_clusters = {0: [], 1: [], 2: [], 3: []}
for act in ACTIVITIES:
    cluster_idx, _ = som_model.find_bmu(act["vector"])
    activity_clusters[cluster_idx].append(act["name"])

return {
    "bp_model": bp_model,
    "bp_history": bp_history,
    "history_01": history_01,
    "history_5": history_5,
    "som_model": som_model,
    "activity_clusters": activity_clusters,
    "norm_params": norm_params
}

# Eksekusi training & caching
cached_data = train_and_cache_models()

# =====
# STREAMLIT USER INTERFACE & LAYOUT
# =====

# Title Banner
st.write("""
<div style="text-align: center; padding: 25px; border-bottom: 2px solid #00e5ff; margin-bottom: 30px;">
    <h1 style="margin: 0; font-size: 42px; font-weight: 800;">THE ZEN-DAY OPTIMIZER</h1>
    <p style="font-size: 18px; color: #a0aec0; margin-top: 10px;">
        Asisten Produktivitas & Pencegahan Burnout Berbasis Jaringan Saraf Tiruan (JST)
    </p>
    <span style="background-color: #00e5ff; color: #000; padding: 5px 12px; border-radius: 20px; font-size: 12px; font-weight: bold; text-transform: uppercase;">
        Projek Akhir Jaringan Saraf Tiruan (JST)
    </span>
</div>
""", unsafe_allow_html=True)

# SIDEBAR: PANEL INPUT USER (REAL-TIME PREDICTION INPUT)
st.sidebar.markdown("""
<div style="background-color: #1a1e30; padding: 15px; border-radius: 8px; border-left: 5px solid #00e5ff; margin-bottom: 20px;">
    <h4 style="margin: 0; color: #ffffff;">Parameter Hari Ini</h4>
    <small style="color: #a0aec0;">Geser nilai di bawah sesuai dengan aktivitas Anda hari ini.</small>
</div>
""", unsafe_allow_html=True)

# Sidebar Inputs for Model 1 (Backpropagation)
sleep_hours = st.sidebar.slider("Durasi & Kualitas Tidur (Jam)", min_value=4.0, max_value=10.0,

```



```

        risk_color = "#ff1744"
        advice = "Bahaya! Tingkat burnout Anda sangat tinggi hari ini. Segera hentikan tugas akademis/pekerjaan berat, matikan layar, lakukan meditasi atau tidur siang."

# Tampilkan dashboard dua kolom
col1, col2 = st.columns([1, 1.2])

with col1:
    st.write("#### Status Kesehatan Mental (Model BP)")
    st.markdown(f"""
<div class="{risk_card_class}">
    <p style="margin: 0; font-size: 16px; color: #cbd5e0; text-transform: uppercase; font-weight: bold;">Prediksi Indeks Burnout</p>
    <h1 style="margin: 10px 0; font-size: 56px; color: {risk_color} !important; font-weight: 800;">{predicted_burnout * 100:.1f}%</h1>
    <p style="margin: 0; font-size: 18px; font-weight: bold; color: {risk_color}">Kategori Risiko: {risk_class}</p>
    <p style="margin: 15px 0 0 0; font-size: 14px; color: #e2e8f0; line-height: 1.6;"><strong>Saran Optimasi:</strong> {advice}</p>
</div>
""", unsafe_allow_html=True)

# Progress Bar visualizer
st.markdown("**Visualisasi Indeks Risiko:**")
st.progress(float(predicted_burnout))

with col2:
    st.write("#### Rekomendasi Aktivitas Harian Anda (Model SOM)")
    st.write("Mencocokkan target vibe harian Anda dengan aktivitas restoratif/produktif menggunakan clustering Kohonen SOM.")

# Cari BMU untuk target vibe yang diinputkan user di sidebar
target_vector = np.array([target_energy, target_social, target_focus])
som_model = cached_data["som_model"]
bmu_idx, distances = som_model.find_bmu(target_vector)

# Cari aktivitas yang terkelompok di cluster yang sama dengan BMU
recommended_activities = cached_data["activity_clusters"][bmu_idx]

# Cari nama label cluster yang representatif
cluster_labels = {
    0: "Zona Istirahat & Pemulihan Pasif (Rest & Passive Recovery)",
    1: "Zona Fokus Sunyi & Refleksi Diri (Quiet Focus & Mindfulness)",
    2: "Zona Interaksi & Relaksasi Sosial (Social & Relaxed Interaction)",
    3: "Zona Produktivitas Aktif & Olahraga (Active Productivity & Exercise)"
}

st.info(f"**Cluster Target Terdeteksi:** {cluster_labels[bmu_idx]}")
st.write("Aktivitas yang disarankan dalam cluster ini:")

if recommended_activities:
    for act_name in recommended_activities:
        st.markdown(f'<div class="activity-badge">{act_name}</div>',
unsafe_allow_html=True)
    else:
        st.write("**Tidak ada aktivitas terdekat. Silakan sesuaikan target vibe di sidebar untuk memperbarui.**")

st.write("---")
# Menampilkan tabel jarak aktivitas untuk transparansi algoritma
with st.expander("Lihat Metrik Kedekatan (Jarak Euclidean) Aktivitas"):
    act_names = [act["name"] for act in ACTIVITIES]
    act_dists = []
    for act in ACTIVITIES:
        dist = np.linalg.norm(act["vector"] - target_vector)

```



```

st.pyplot(fig2)

st.write("---")
st.write("#### Peta Klusterisasi Kohonen Self-Organizing Map (Grid 2x2)")
st.write("Visualisasi ini menggambarkan bagaimana 10 aktivitas di petakan ke dalam 4 neuron
grid output (2x2) berdasarkan kedekatan fitur bobotnya.")

# Buat plot peta grid SOM
fig3, ax3 = plt.subplots(figsize=(10, 5))
fig3.patch.set_facecolor('#0f111a')
ax3.set_facecolor('#161925')

# Warna latar belakang masing-masing sel grid untuk identifikasi estetis
grid_colors = [
    ['#1a1e35', '#241a35'],
    ['#1a2a35', '#1a3528']
]

ax3.set_xlim(-0.5, 1.5)
ax3.set_ylim(-0.5, 1.5)
ax3.set_xticks([0, 1])
ax3.set_yticks([0, 1])
ax3.set_xticklabels(['Kolom 0', 'Kolom 1'], color='ffffff')
ax3.set_yticklabels(['Baris 0', 'Baris 1'], color='ffffff')
ax3.grid(True, color='#2c313d', linestyle='-', linewidth=2)

# Judul representatif untuk grid neuron
grid_labels = {
    0: "Rest & Recovery\n(Node [0,0])",
    1: "Quiet Focus\n(Node [0,1])",
    2: "Social Vibe\n(Node [1,0])",
    3: "Active Productivity\n(Node [1,1])"
}

# Tampilkan teks aktivitas pada sel grid masing-masing
for idx in range(4):
    r, c = cached_data["som_model"].get_grid_coords(idx)
    acts = cached_data["activity_clusters"][idx]

    # Gambar kotak sel
    rect = plt.Rectangle((c-0.45, r-0.45), 0.9, 0.9, facecolor=grid_colors[r][c],
edgecolor='#00e5ff', linewidth=1.5, alpha=0.7)
    ax3.add_patch(rect)

    # Tambahkan judul node
    ax3.text(c, r+0.35, grid_labels[idx], color='#00e5ff', fontsize=11, fontweight='bold',
ha='center', va='center')

    # Tambahkan daftar aktivitas
    acts_text = "\n".join([f"• {act.split(' ')[0]}" for act in acts]) if acts else "Kosong"
    ax3.text(c, r-0.08, acts_text, color='#e2e8f0', fontsize=9.5, ha='center', va='center',
wrap=True)

ax3.set_title("Self-Organizing Map Topographic Activation Mapping", color='ffffff',
fontsize=14, pad=15)
for spine in ax3.spines.values():
    spine.set_color('#2c313d')

st.pyplot(fig3)

# =====
# TAB 3: ACADEMIC DOCUMENTATION FORMAT (INDONESIAN REPORT BAB I-V)
# =====
with tab_academic:

```



```

1. **Layer Input (3 Neuron)**:
  - Kebutuhan Energi Fisik (0 s/d 1)
  - Tingkat Interaksi Sosial (0 s/d 1)
  - Kebutuhan Fokus Mental (0 s/d 1)
2. **Layer Output (Grid 2D 2x2 = 4 Neuron)**:
  - Direpresentasikan sebagai koordinat grid  $[r, c]$  di mana  $r \in \{0,1\}$  dan  $c \in \{0,1\}$ .
  - Setiap node dalam grid memiliki vektor bobot berdimensi 3 ( $W_j = [w_{j1}, w_{j2}, w_{j3}]$ ).
  - Metrik kemiripan yang digunakan untuk mencari neuron terdekat (*Best Matching Unit* / BMU) adalah **Jarak Euclidean**:
    
$$D(j) = \sqrt{\sum_{i=1}^3 (x_i - w_{ji})^2}$$

    """)

```

```

elif chapter.startswith("Bab III:"):
    st.markdown(r"""
    ### BAB III: CONTOH ITERASI MANUAL

```

Di bawah ini adalah simulasi numerik satu langkah iterasi maju (Forward) dan satu langkah iterasi mundur (Backward) pada model Backpropagation dengan arsitektur yang disederhanakan (2 Input, 2 Hidden, 1 Output) untuk memudahkan perhitungan manual.

```

#### 3.1 Parameter Awal
- **Input**:  $x_1 = 0.8$ ,  $x_2 = 0.4$ 
- **Target Output**:  $t = 0.1$ 
- **Learning Rate**:  $\eta = 0.5$ 

**Bobot Layer Tersembunyi ( $W_1$  &  $b_1$ ):**
- Ke Hidden 1 ( $h_1$ ):  $w_{11} = 0.15$ ,  $w_{21} = 0.25$ , Bias  $b_{11} = 0.35$ 
- Ke Hidden 2 ( $h_2$ ):  $w_{12} = 0.20$ ,  $w_{22} = 0.30$ , Bias  $b_{12} = 0.35$ 

**Bobot Layer Output ( $W_2$  &  $b_2$ ):**
- Dari  $h_1$  ke output  $o_1$ :  $w_{out1} = 0.40$ 
- Dari  $h_2$  ke output  $o_1$ :  $w_{out2} = 0.45$ 
- Bias Output  $b_2 = 0.60$ 

---

#### 3.2 Perambatan Maju (Forward Pass)

**Langkah 1: Hitung Net input dan aktivasi pada Layer Tersembunyi**
- Net Input  $h_1$  ( $z_{h1}$ ):
  
$$z_{h1} = x_1 \cdot w_{11} + x_2 \cdot w_{21} + b_{11}$$

  
$$z_{h1} = (0.8 \cdot 0.15) + (0.4 \cdot 0.25) + 0.35 = 0.12 + 0.10 + 0.35 = 0.57$$

- Aktivasi  $h_1$  ( $a_{h1}$ ):
  
$$a_{h1} = \frac{1}{1 + e^{-0.57}} = \frac{1}{1 + 0.5655} \approx 0.6387$$


- Net Input  $h_2$  ( $z_{h2}$ ):
  
$$z_{h2} = x_1 \cdot w_{12} + x_2 \cdot w_{22} + b_{12}$$

  
$$z_{h2} = (0.8 \cdot 0.20) + (0.4 \cdot 0.30) + 0.35 = 0.16 + 0.12 + 0.35 = 0.63$$

- Aktivasi  $h_2$  ( $a_{h2}$ ):
  
$$a_{h2} = \frac{1}{1 + e^{-0.63}} = \frac{1}{1 + 0.5326} \approx 0.6525$$


**Langkah 2: Hitung Net input dan aktivasi pada Layer Output**
- Net Input Output  $o_1$  ( $z_{o1}$ ):
  
$$z_{o1} = a_{h1} \cdot w_{out1} + a_{h2} \cdot w_{out2} + b_2$$

  
$$z_{o1} = (0.6387 \cdot 0.40) + (0.6525 \cdot 0.45) + 0.60 = 0.2555 + 0.2936 + 0.60 = 1.1491$$

- Aktivasi Output  $o_1$  ( $a_{o1}$  / Output Aktual):
  
$$a_{o1} = \frac{1}{1 + e^{-1.1491}} = \frac{1}{1 + 0.3169} \approx 0.7593$$


**Langkah 3: Hitung Error (Loss)**
- Mean Squared Error untuk sampel tunggal:
  
$$E = \frac{1}{2}(t - a_{o1})^2 = \frac{1}{2}(0.1 - 0.7593)^2 = \frac{1}{2}(-0.6593)^2 \approx 0.2173$$


```


Berdasarkan visualisasi grafik pada Tab **"Visualisasi Model"**, dilakukan eksperimen untuk menganalisis pengaruh perubahan parameter ***Learning Rate* (η)** terhadap laju konvergensi dan kestabilan model Backpropagation:

1. ***Learning Rate Sangat Kecil ($\eta = 0.01$)**:**

 - ***Pengaruh**:** Penurunan nilai MSE berjalan sangat lambat. Memerlukan jumlah epoch yang jauh lebih banyak untuk mencapai konvergensi optimal.
 - ***Kelebihan**:** Sangat stabil dan minim risiko berosilasi di sekitar nilai minimum lokal.

2. ***Learning Rate Optimal ($\eta = 0.20$)**:**

 - ***Pengaruh**:** Model mencapai konvergensi yang cepat dan halus. Dalam sekitar 600-800 epoch, MSE sudah mulai mendatar di angka yang sangat rendah (< 0.008).
 - ***Kelebihan**:** Efisiensi komputasi tinggi tanpa mengorbankan akurasi klasifikasi.

3. ***Learning Rate Agresif/Besar ($\eta = 0.50$)**:**

 - ***Pengaruh**:** Terjadi percepatan penurunan MSE pada epoch-epoch awal. Namun, grafik terlihat berosilasi lebih tajam dan rentan melompati (**overshooting**) nilai minimum global yang sebenarnya.

****Rekomendasi Parameter**:**

Untuk dataset optimasi burnout ini, kombinasi paling efisien adalah **$\eta = 0.2$** dengan **Epoch = 1000 hingga 1200**.

"""

elif chapter.startswith("Bab V:"):

st.markdown("""

BAB V: KESIMPULAN & UJI SOLUSI

5.1 Kesimpulan Keberhasilan Model

Berdasarkan pengujian sistem secara real-time pada dashboard utama:

1. ***Model Backpropagation*** berhasil memetakan variabel-variabel input harian (Tidur, Tugas, Layar, dan Sosial) menjadi indikator persentase burnout yang relevan secara logis dan ilmiah. Model mampu mengidentifikasi skenario ekstrem (misalnya, tidur 4 jam dengan 10 tugas menghasilkan burnout $> 80\%$).

2. ***Model Kohonen SOM*** berhasil membagi 10 aktivitas harian yang variatif menjadi 4 cluster topografi yang teratur secara mandiri (**unsupervised**). Aktivitas pasif terkelompok dekat satu sama lain, begitupun aktivitas sosial dan akademis terfokus.

3. ***Integrasi Solusi*** berhasil menjembatani prediksi resiko kesehatan mental dengan solusi pemulihan aktivitas harian secara real-time.

5.2 Saran Pengembangan

1. ***Ekspansi Dataset**:** Menggunakan dataset riil dari log aktivitas smartwatch atau aplikasi pelacak waktu guna meningkatkan validitas prediksi.

2. ***Struktur Hidden Layer**:** Uji coba penambahan jumlah hidden layer (Deep Neural Network) untuk pola hubungan yang lebih kompleks.

""")

=====

FOOTER

=====

st.write("---")

st.write("""

<div style="text-align: center; color: #a0aec0; padding: 15px; font-size: 13px;">

Dibuat oleh Tim JST - Final Exam Project "The Zen-Day Optimizer" © 2026. Built with Streamlit and NumPy.

</div>

""", unsafe_allow_html=True)

C. Library Tambahan yang Digunakan (requirements.txt)

```
numpy>=1.20.0  
streamlit>=1.20.0  
matplotlib>=3.5.0  
pandas>=1.3.0
```